

5. Phase 2 ADS XBRL GL構文結合：環境定義、変換操作及び関数単位の処理

この文書は、**src/syntax_binding_ads_xbri_gl.py** を対象とし、Phase 1構造化CSVからADS XBRL GLインスタンスを生成する構文結合処理を独立して説明します。

この処理は **semantic_path** で元の構造化CSV列及び行スコープを選択し、XBRL GLの **xpath**、**selector**、**element**、**unitRef**及び**decimals**規則に従ってXMLを構築します。そのため、フラットファイルを生成する **semantic_binding.py** とは分けて扱います。

処理の流れ

```
Structured CSV + ADS XBRL GL syntax binding
→ 結合定義及び通貨小数桁表の読み込み
→ 階層行スコープの検証
→ 文書 / VAT / 明細行の選択
→ XPathとselectorに基づくタプルコンテナの生成
→ item、context、unit、documentInfoの生成
→ XBRL GL要素順の整列及びXBRL出力
```

環境及び結合定義

結合ファイル及びレビュー用ワークブック

specs/bindings/syntax/ 以下の6種類の構文結合CSVで、次の対象を定義します。

- Invoices Received
- Invoices Generated
- Invoices Received Lines
- Invoices Generated Lines
- Supplier Listing
- Customer Master

レビュー用ワークブックは、次のとおりです。

```
specs/bindings/ADS_XBRL_GL_Bindings.xlsx
```

変換操作の例

チュートリアル実行コマンド

```
& $python .\src\tutorial\03_generate_ads_xbrl_gl.py
```

出力ディレクトリ：

```
out/tutorial/xbrl-gl/
```

ラッパースクリプトは、次のファイルを使用します。

```
src/syntax_binding_ads_xbrl_gl.py  
specs/bindings/syntax/ADS_Invoices_Received_XBRL_GL_Binding.csv  
specs/lhm/EN16931_CIUS_Invoice_LHM.csv  
specs/Currency.csv
```

生成されるインスタンスには、XBRLコンテキスト及び単位に加え、対象ビューに必要なXBRL GLのタプル階層が含まれます。

変換仕様

対象範囲及びセレクトアの要件

ADS XBRL GLインスタンスは、**src/syntax_binding_ads_xbrl_gl.py** で生成します。ADS PSVで使用するものと同じセマンティックパス及びクラススコープを、6種類のXBRL GL対象ビューにも使用します。生成プログラムは、選択した結合定義に従って、コンテキスト、単位、文書情報、セレクトア及び対象ファクトを作成します。

売手の識別情報には、**identifierType=V** である識別子参照セレクトアを使用します。売手の郵便住所ファクトは、対応する **identifierAddress** コンテナの下に配置します。結合表の統合処理及びXBRLの構築処理では、このセレクトアの祖先関係を保持しなければなりません。保持されない場合、Phase 1に住所が存在していても、売手一覧には住所が出力されません。

主要関数

- **load_bindings** : ADS XBRL GL結合行を読み込み、正規化します。
- **validate_hierarchical_row_scopes** : Phase 1と同じ親行と繰返し子行の分離規則を適用します。
- **rows_for_binding** : 各セマンティックファクトの元データ行を選択します。
- **unit_ref_from_rule** 及び **decimals_for_item** : XBRLの単位及び小数精度を導出します。
- **path_parts**、**parse_path_step** 及び **selector_matches** : 対象XBRL GLパス及びセレクトア述語を解釈します。
- **container_for_path** 及び **ensure_child** : 識別子参照の住所コンテナを含め、選択された対象階層を正確に作成します。
- **append_item** : 空の数値を出力せずにファクトを作成します。

- **build_instance** : 一つの対象ビューについて、コンテキスト、単位、文書情報及び結合済みファクトを作成します。
- **reorder_tree** : 安定した対象順序で子要素を直列化します。

関数単位の実装解説

Phase 2構文結合 : 構造化CSVからADS XBRL GLへ

スクリプト :

```
src/syntax_binding_ads_xbri_gl.py
```

主な関数 :

```
load_bindings(...)  
validate_hierarchical_row_scopes(...)  
build_instance(...)  
convert_file(...)
```

入力 :

- Phase 1構造化CSV
- ADS XBRL GL構文結合CSV
- 通貨の小数桁数表 **specs/Currency.csv**

出力 :

次の六つのADS XBRL GLタプルビューのいずれかを生成します。

- **Invoices_Received.xbri**
- **Invoices_Generated.xbri**
- **Invoices_Received_Lines.xbri**
- **Invoices_Generated_Lines.xbri**
- **Supplier_Listing.xbri**
- **Customer_Master.xbri**

結合定義の解釈

ADS XBRL GL結合定義は、**semantic_path** と **structured_csv_column** により、入力構造化CSVの値を選択します。**element** と **xpath** により、出力先XBRL GLの項目及びタプル位置を識別します。

XBRL GLインスタンスを構築する前に、**validate_hierarchical_row_scopes(...)** が、結合表の **type=C**、**multiplicity** 及び **semantic_path** から各ファクトの所有ディメンションを決定します。非繰返しの子は、最も近い繰返し祖先の行に属します。繰返しの子は、1番目から独立した行を所有します。親ファクトと繰返し子ファクトが同じ子行に設定されている入力は拒否します。

結合定義辞書の例：

```
{
  "source_column": "InvoiceLineIdentifier",
  "semantic_path": "$.invoice.invoiceLine.invoiceLineIdentifier",
  "element": "gl-cor:documentNumber",
  "xpath": "/xbrli:xbrl/gl-cor:accountingEntries/...",
  "value_source": "structured_csv_column",
  "unit_ref_rule": "",
  "decimals": ""
}
```

`syntax_binding.py` と異なり、このスクリプトは結合定義行を辞書のまま保持します。

```
bindings: list[dict[str, str]]
rows: list[dict[str, str]]
currency_minor_units: dict[str, str]
```

ADSディメンションの所有関係及び入力検証

`load_bindings(binding_csv)` は、XBRL生成と構造化CSV検証の両方に必要な次の結合属性を保持します。

- **type** : クラス行 (C) とファクト行 (A) を区別する。
- **multiplicity** : クラスが繰り返すかどうかを決定する。
- **semantic_path** : クラスの祖先関係を定義する。
- **structured_csv_column** : **source_column** となる。
- **element** 及び **xpath** : XBRL GLの出力先を識別する。

`validate_hierarchical_row_scopes(rows, bindings)` は、次を行います。

1. **type=C** 行を調べる。請求書ルートと、**multiplicity_repeats(...)** が真となるクラスを **class_dimensions** に登録する。
2. **dimension_name(source_column)** が、**InvoiceLine** などのクラス列を **dInvoiceLine** へ変換する。
3. 各 **type=A** ファクトについて、登録済みクラスディメンションが見つかるまで、セマンティックパスの右端からセグメントを除去する。結果を **field_dimensions** に保存する。
4. 構造化CSVの各行について、値が入っている最も深い **dXxx** 列を取得する。
5. ADSへ結合する値が入っているファクトの所有ディメンションが、その行のディメンションと異なる場合は拒否する。

したがって、単一の子クラスは独立した行スコープを持たず、最も近い繰返し祖先の行に属します。繰返し子は、1番目から自身の行スコープを持ちます。親ファクトと繰返し子ファクトを、最初の子行と一緒に設定することはできません。

この検証は、**convert_file(...)** が **read_csv(...)** の直後、**build_instance(...)** の前に呼び出します。そのため、階層が不正な入力からXBRL GLインスタンスの一部だけが生成されることはありません。

ADS入力行の選択

検証後、**rows_for_binding(rows, binding)** が各ファクト結合で使用する疎な入力行を選択します。

- VAT内訳のファクトは、**dVatBreakdown** が設定された行を必要とする。
- Invoice Lineのファクトは、**dInvoiceLine** が設定された行を必要とする。
- 文書レベルのファクトは、入力値が最初に現れる行を使用する。

first_non_empty(...) は、文書全体に共通する通貨、日付及び主体の値を取得します。選択した入力行は **unit_ref_from_rule(...)** にも渡されるため、金額ファクトは結合された構造化CSVの通貨列から単位を取得できます。

XBRL XMLの構築

build_instance(...) は、**xbrli:xbrl** をルートとする **ET.Element** ツリーを作成します。コンテキスト、単位、XBRL GL文書メタデータ及び選択した結合パスに必要なタプルコンテナを追加します。請求書の結合では文書構造を作成し、Supplier Listing及びCustomer Masterでは該当する識別子参照構造を作成します。

パス処理：

- **path_parts(xpath)**：出力先XPathをステップに分割する。
- **parse_path_step(step)**：要素名とセレクト条件を分離する。
- **container_for_path(root, xpath, currency, monetary_decimals)**：タプルパスをたどる。
- **ensure_child(...)**：タプルコンテナを検索又は作成する。
- **selector_matches(...)**：既存要素がセレクトの子要素値を満たすか確認する。
- **append_item(...)**：末端の項目要素を作成して値を書き込む。

Supplier Listingの結合では、**gl-cor:identifierReference[gl-cor:identifierType="V"]** により仕入先識別子を選択します。Sellerの郵便住所ファクトは、その識別子参照配下の出力先パスを使用し、**gl-bus:identifierAddress** と、その子要素 **identifierStreet**、**identifierAddressStreet2**、**identifierCity**、**identifierStateOrProvince**、**identifierCountry**、**identifierZipOrPostalCode** を作成します。セレクトは仕入先タプルを識別するだけであり、対応するファクト行が結合表に存在しない限り、住所の子要素は作成しません。

Seller住所を **gl-bus:organizationAddress** に書き込んではいけません。このタプルは、**entityInformation** 配下にある報告主体の住所を表します。仕入先をvendor identifier referenceで表す場合は、同じ **identifierType=V** のタプル配下に **gl-bus:identifierAddress** を使用します。

rows_for_binding(rows, binding) が入力行を選択します。Invoice Lineの結合では、対象の入力列と明細ディメンションの両方が設定された行を選びます。文書レベルの結合では文書行を選びます。

数値ファクトでは、次を使用します。

- **unit_ref_from_rule(...)** : **unitRef** を決定する。
- **decimals_for_item(...)** : **decimals** を決定する。
- **load_currency_minor_units(...)** : **specs/Currency.csv** から通貨の小数桁数を取得する。
- **add_missing_units(...)** : 参照されたすべての単位が宣言されていることを確保する。

構築処理は、次の順序で完了します。

1. **make_xbri_root(...)** が **xbri:xbri** と **schemaRef** を作成する。
2. **add_context_and_units(...)** が既定コンテキスト及び初期単位を追加する。
3. **add_document_info(...)** が必須のXBRL GL文書メタデータを書き込む。
4. **build_instance(...)** がVATの各出現及びその他の結合ファクトを書き込む。
5. **reorder_tree(...)** が対応するXBRL GL子要素順を適用する。これには、**identifierReference** 配下の **identifierAddress** と、住所ファクトのスキーマ順が含まれる。
6. **add_missing_units(...)** が生成したファクトで参照される単位を宣言する。
7. **convert_file(...)** がインデントを整え、XML文書を書き出す。
8. **main(...)** が **input_files(...)** により単一ファイル又はディレクトリを解決し、**binding_target_stem(...)** と **schema_href_for_output(...)** により出力ファイル名を決定する。

回帰テスト

回帰テストの概要

test_ads_*_xbri_gl.py という名前の対象別スクリプトをすべて実行した後、次のテストを実行します。

```
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

完全なテストコマンド

次のテストは、最初にPhase 1構造化CSVを再生成し、その後、**specs/bindings/syntax/** 以下のADS XBRL GL構文結合CSVを使用します。

```
& $python .\tests\test_ads_invoices_received_xbri_gl.py
& $python .\tests\test_ads_invoices_generated_xbri_gl.py
& $python .\tests\test_ads_invoices_received_lines_xbri_gl.py
& $python .\tests\test_ads_invoices_generated_lines_xbri_gl.py
& $python .\tests\test_ads_supplier_listing_xbri_gl.py
& $python .\tests\test_ads_customer_master_xbri_gl.py
```

想定出力 :

```
out/phase2/ADS_XBRL_GL/<structured-csv-stem>/<target-view>.xbri
```

対象ビューファイル：

```
Invoices_Received.xbrl  
Invoices_Generated.xbrl  
Invoices_Received_Lines.xbrl  
Invoices_Generated_Lines.xbrl  
Supplier_Listing.xbrl  
Customer_Master.xbrl
```

test_ads_supplier_listing_xbrl_gl.py は、売手の名称及び識別子が **identifierType=V** の識別子参照へ書き込まれ、売手の番地、市区町村、国及び郵便番号が、その **gl-bus:identifierAddress** タプルの下へ書き込まれることを確認します。